

Manual Espetacular de Arquitetura: Agentes & Skills

Descentralização de Memória e Padrões de Alta Performance para a Antigravity Engine

Este documento serve como guia definitivo para corrigir os vícios de acoplamento rígido de contexto e elevar o padrão de engenharia na construção de fluxos autônomos dentro do diretório `.agents/`. Abaixo, detalhamos a separação cirúrgica entre **Workflows (Agentes)** e **Skills (Capacidades)**.

1. O Paradigma de Separação de Conceitos

A arquitetura da Antigravity Engine apoia-se estritamente na **Memória Descentralizada**. Agentes monolíticos estruturados em arquivos gigantescos saturam a janela de contexto do LLM com instruções redundantes, provocando degradação nas tomadas de decisão e desperdício de tokens.

Componente	Responsabilidade Principal	Foco Operacional	Localização Nativa
Workflow (Agente)	Orquestração sequencial e governança de processos.	O que fazer (Fluxo de Etapas).	<code>.agents/workflows/</code>
Skill (Habilidade)	Capacidades técnicas modulares, templates e scripts.	Como fazer (Especialidade).	<code>.agents/skills/[nome]/</code>

💡 Regra de Ouro da Performance

O Workflow nunca deve conter regras de codificação, e a Skill nunca deve ditar etapas de orquestração global. As ferramentas e templates só entram no contexto do LLM sob demanda, quando a skill é explicitamente evocada.

2. Anatomia de uma Skill Profissional

Abaixo apresenta-se a topologia padrão de uma pasta de skill autocontida e expansível:

```
.agents/skills/my-skill/
├── SKILL.md           # Diretrizes maestras e gatilhos de ativação (Obrigatório)
├── scripts/           # Automações locais e utilitários auxiliares (Opcional)
├── examples/          # Demonstrações realistas de saída (Few-shot learning) (Opcional)
└── resources/         # Modelos brutos, checklists e JSON/YAML schemas (Opcional)
```

3. Arquivos e Templates Práticos

Utilizando o agente de engenharia de software (**TDD Green Phase / Implementação**) como modelo unificado, seguem as especificações exatas de cada artefato.

3.1. Template do Workflow (O Agente Processual)

Este arquivo mapeia o comando de barra e dita o pipeline de tarefas sem poluir o modelo com regras de Clean Code.

```
---
title: "Implementation Engineer Agent"
description: "Orquestra a Fase Green do ciclo TDD escrevendo o código de produção mínimo."
```

```

---

# Agent: Implementation Engineer (`/codigo`)

Você atua como o engenheiro executor focado em fazer a suíte de testes passar. Sempre responda em português.

## 🚀 Fluxo de Execução

1. **Pre-flight Check**: Execute o comando local para capturar a branch git ativa.
2. **Resolução de Contexto**: Identifique o arquivo de especificação técnica sob `01-concepcao/sdd-[slug].md`.
3. **Ativação da Skill**: Invoque a capacidade técnica `@codigo` para herdar os rigorosos padrões de escrita e uso de templates.
4. **Entrega**: Apresente as alterações de código acompanhadas de um bloco bash isolado para execução dos testes unitários. Se a suíte retornar 100% verde, direcione para o passo `/refatorar`.

```

3.2. Template de Instruções Principais (`SKILL.md`)

Fica localizado em `.agents/skills/codigo/SKILL.md`. Carrega a persona técnica e os links relativos para seus recursos internos.

```

---
name: "codigo"
description: "Garante a escrita de código de produção mínimo usando SOLID, Type Hints e gestão de pendências."
---

# Skill: TDD Green Phase & Production Coding

Esta skill provê as regras fundamentais de engenharia de software para transformar falhas de testes em sucesso funcional.

## ⚙️ Regras Universais de Engenharia
- **Código Mínimo Consecutivo**: Escreva unicamente a lógica necessária para satisfazer as asserções atuais. Proibido código morto ou funcionalidades futuras.
- **Padrão Estrito de Tipagem**: Todas as novas assinaturas requerem Type Hints nativos.
- **Acoplamento à Documentação**: Toda classe ou função gerada deve referenciar sua respectiva nota conceitual através da tag `Ref: Obsidian note [[nome-da-nota]]`.

## 📁 Recursos e Dependências da Skill
- **Template de Tarefas**: Use o modelo interativo localizado em `resources/task_template.md` para criar a `task_list.md`.
- **Implementação de Referência**: Siga rigorosamente a semântica do exemplo contido em `examples/docstring_example.py` para padronizar documentações.

```

3.3. Recursos Auxiliares (`resources/task\_template.md`)

Modelo bruto que o agente usará para instanciar a listagem interativa na interface do usuário.

```

# Lista de Tarefas de Implementação (`task.md`)

- [ ] **Task 1**: Criar componente estrutural `[NOME]` em `[CAMINHO]`
  - **Status**: Pendente
  - **Foco de Teste**: `test_[NOME_DO_TESTE]`
- [ ] **Task 2**: Estruturar tratamento de exceções corporativas

```

- **Status:** Pendente
- **Foco de Teste:** `test\_[EXCECAO]`

3.4. Implementações de Referência (`examples/docstring\_example.py`)

O arquivo para aprendizado por exemplo (few-shot), permitindo que o LLM copie a estrutura ideal sem precisar ler longos textos explicativos.

```
def processar_pagamento_fatura(valor_total: float, identificador_id: str) -> bool:
    """Executa a liquidação financeira de uma fatura de serviços.

    Args:
        valor_total: Valor monetário flutuante da transação.
        identificador_id: String uuid única da fatura alvo.

    Returns:
        Booleano indicando o sucesso da transação bancária.

    Raises:
        PaymentGatewayException: Se houver indisponibilidade do provedor externo.

    Domain Context:
        Aplica a regra de negócio financeira BR-101 (Liquidação Direta).
        Ref: Obsidian note [[2026-financial-rules]]
    """
    # Código funcional mínimo exigido pelo teste
    return True
```

3.5. Scripts de Automação (`scripts/verify\_coverage.py`)

Exemplo de script utilitário opcional que a própria skill pode invocar nos bastidores para agilizar auditorias de código.

```
import json
import sys

def checar_cobertura(caminho_json):
    with open(caminho_json, 'r') as f:
        dados = json.load(f)
    cobertura_total = dados['totals']['percent_covered']
    if cobertura_total < 90.0:
        print(f"⚠ Falha: Cobertura de {cobertura_total}% está abaixo da meta de 90%.")
        sys.exit(1)
    print(f"✅ Sucesso: Cobertura de {cobertura_total}% validada.")
    sys.exit(0)

if __name__ == "__main__":
    checar_cobertura(sys.argv[1])
```

4. Boas Práticas e Higiene de Contexto

- **Caminhos Sempre Relativos à Pasta da Skill:** Quando fizer menção a assets de recursos dentro de `SKILL.md`, utilize links curtos (ex: `resources/template.md`). A engine resolve automaticamente o path absoluto de acordo com o escopo da execução.
- **Evite Duplicar Parágrafos:** Se uma regra de qualidade já está expressa na skill, apague-a completamente do manifesto do workflow. O workflow chama a skill, a skill injeta o comportamento.

- **Uso Extensivo de Exemplos (Few-Shot):** O modelo aprende melhor mimetizando arquivos reais na pasta `examples/` do que lendo manuais enfadonhos de regras gramaticais ou restrições abstratas.